

Prevent Concurrent User Update Conflicts

1. Problem Statement

When multiple users edit the same shared resource simultaneously:

- One user overwrites another
- Data inconsistency occurs
- Users lose work
- Bad UX happens

This is called:

Concurrent update conflict / Lost update problem

2. Real Examples

Hotel Booking

Two users book same room simultaneously.

CMS / Admin Panel

Two admins edit same homepage banner.

Push Notification Tool

Two marketers edit same campaign title.

Shared Config Panel

Two engineers edit same feature flag.

3. Goal

Ensure:

- Only one user edits critical resource at a time
OR
- Detect conflict before save

4. Common Solutions

Method	Best For
Pessimistic Locking	Critical exclusive edits
Optimistic Locking	Low conflict environments
Queueing	Serialized updates
CRDT / Merge	Collaborative editing

5. Solution in Our Example = Pessimistic Locking

When user clicks **Edit**:

System locks resource for a limited time.

6. Example DB Schema

SQL

```
Configurations(  
    config_id PK,  
    content,  
    locked_until,  
    edit_username,  
    edit_email  
)
```

7. Workflow

Step 1: Alice clicks Edit

None

```
PUT /configs/{id}/lock
```

Backend checks:

SQL

```
SELECT locked_until  
  
FROM configurations  
  
WHERE config_id = 1;
```

If expired/unlocked:

SQL

```
UPDATE configurations  
  
SET locked_until = now + 10 min,  
    edit_username = 'Alice'  
  
WHERE config_id = 1  
  
AND locked_until = old_value;
```

Step 2: Alice Gets Lock

Response:

```
JSON
{
  "can_edit": true
}
```

Step 3: Bob clicks Edit

Same request.

But update fails because lock already changed.

Response:

```
JSON
{
  "can_edit": false,
  "editor": "Alice",
  "retry_after": "8 minutes"
}
```

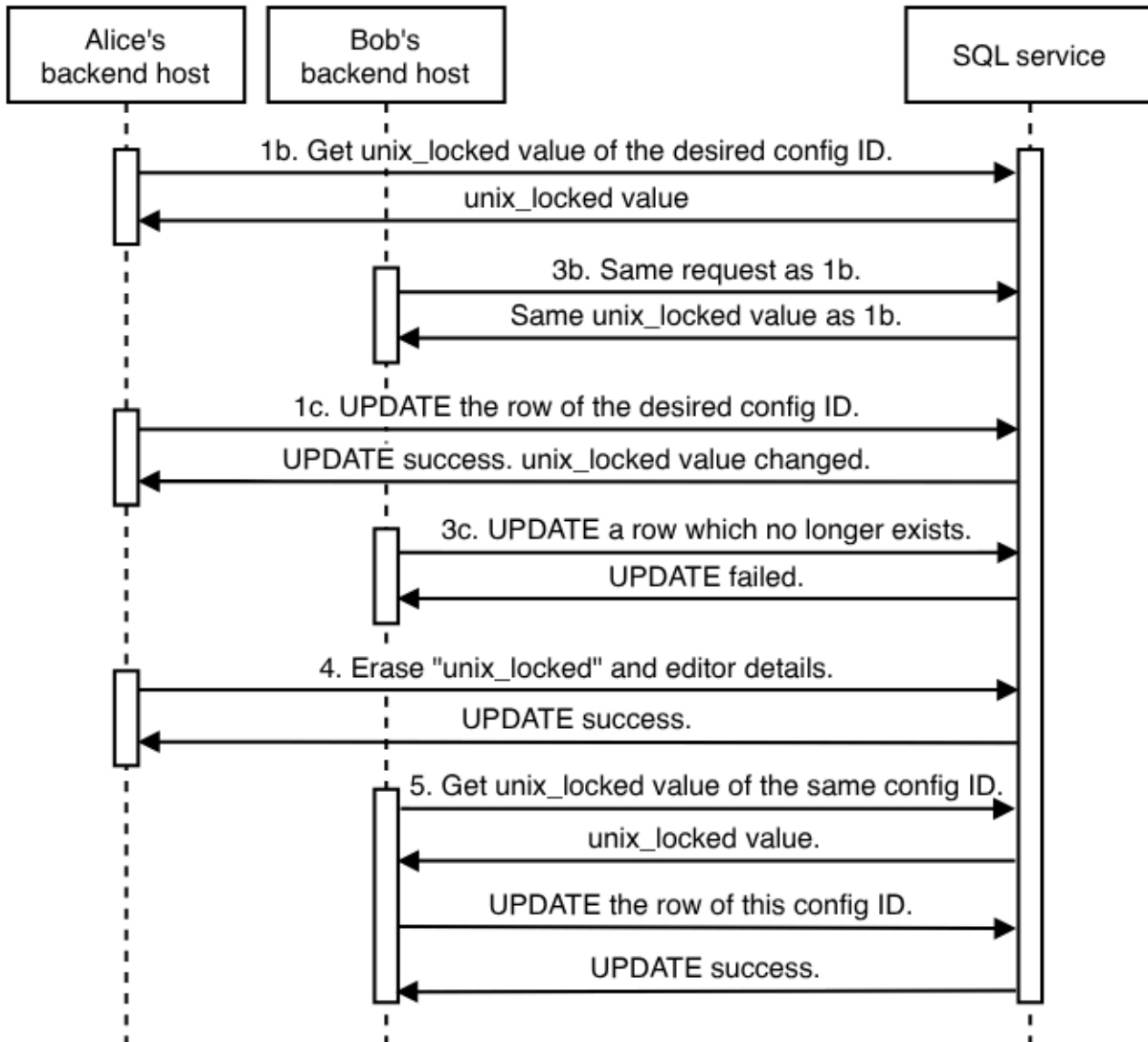
Step 4: Alice Saves

None

```
PUT /configs/{id}
```

System:

- Saves content
- Releases lock



8. Why Conditional UPDATE is Important

This avoids race conditions.

SQL

```
WHERE locked_until = old_value
```

Ensures only one update wins.

This is similar to **Compare-And-Swap (CAS)**.

9. Why Timeout Needed?

If Alice closes browser or crashes:

Without timeout = permanent lock.

So lock auto-expires after:

- 5 min
- 10 min
- configurable TTL

10. UX Improvements

While Locked:

- Disable Edit button
- Show current editor
- Show countdown timer

Example:

Alice is editing. Try again in 8 minutes.

Great product thinking.

11. Important Distributed Systems Concern

Different servers may have clock drift.

Use:

- DB server timestamp
- Central time source
- Extra safety buffer

12. Better Alternatives

A. Optimistic Locking (Preferred Often)

Use version column.

```
SQL
```

```
version = 7
```

Alice reads version 7.

Bob reads version 7.

Alice saves → version becomes 8.

Bob saves with version 7 → rejected.

Best For:

- Rare conflicts
- Better UX
- No hard locks

B. Distributed Lock (Redis)

Use:

- Redis SETNX
- TTL lock

Useful across multiple app servers.

C. Collaborative Editing

For docs like Google Docs:

- Operational Transform
- CRDT

Multiple users edit simultaneously.

13. Drawbacks of Pessimistic Locking

- Users blocked
- Lock timeout frustration
- Stale locks
- Requires lock cleanup

14. When to Use Which

Scenario	Best Choice
Hotel room booking	Transaction + row lock
Admin config	Pessimistic lock
Blog edit	Optimistic lock
Google Docs	CRDT

15. Hotel Booking Better Design

For inventory booking:

Use atomic SQL transaction:

SQL

```
UPDATE rooms
```

```
SET available = available - 1
```

```
WHERE room_id=1
```

```
AND available > 0;
```

Prevents oversell.

16. Interview Script

Say:

“For low-frequency but costly conflicts, I’d use pessimistic locking with TTL and owner metadata. For common edits with low collision probability, I’d prefer optimistic locking using version numbers.”

That sounds excellent.

17. Final Summary

- Concurrent edits cause lost updates
- Use locking or versioning
- Pessimistic lock = exclusive edit access
- Optimistic lock = conflict on save
- Add TTL + UX messaging
- Choose solution based on conflict frequency

18. Memorable Line

If two users can edit the same thing, concurrency is already your problem.